

Guía Práctica N° 4

Fundamentos de Redes Neuronales

Introducción a Redes Neuronales y Arquitectura

1. El Perceptrón original generó grandes expectativas en sus inicios, pero luego se demostró que tenía limitaciones matemáticas fundamentales. Utilizando el caso del operador lógico XOR, explique cuál es esta limitación y por qué una única neurona con una función de activación tipo escalón (Heaviside) no puede separar este espacio.
2. Describa qué elementos estructurales diferencian a un Perceptrón Multicapa (MLP) de un Perceptrón simple. ¿Qué papel juegan las “capas ocultas” y por qué se considera que la existencia de estas capas habilita el concepto de Aprendizaje Profundo (*Deep Learning*)?

Funciones de Activación

3. Si construimos una Red Neuronal Profunda con múltiples capas ocultas, pero omitimos incluir funciones de activación no lineales entre ellas (es decir, permitimos únicamente combinaciones lineales), ¿qué impacto tendrá esto en la capacidad de la red para aprender? Justifique matemáticamente.
4. Durante mucho tiempo, la función Sigmoidea y la Tangente Hiperbólica (TanH) fueron las opciones por defecto para las capas ocultas. Sin embargo, en redes muy profundas suelen presentar el problema de gradientes que se desvanecen (*Vanishing Gradients*). Explique intuitivamente en qué consiste este fenómeno basándose en la forma de las funciones.
5. La función ReLU (*Rectified Linear Unit*) se popularizó en gran medida porque mitiga el problema de los gradientes desvanecientes al mantener una derivada igual a 1 en su parte positiva. Describa en qué consiste el nuevo problema de las neuronas muertas (*Dying ReLU*) que surge con esta función y cómo variantes como Leaky ReLU o SiLU (Swish) intentan solucionarlo.
6. En problemas de clasificación multiclase, es estándar utilizar la función Softmax en la última capa de la red porque transforma todo el vector de salidas en probabilidades. Explique matemáticamente por qué las salidas suman siempre 1 y qué desventaja podría tener utilizar múltiples funciones Sigmoideas independientes para este tipo de problemas.

Forward y Backward Propagation

7. En el modo de entrenamiento de una red neuronal, el propósito es optimizar los parámetros utilizando tanto *Forward* como *Backward Propagation*. Defina brevemente qué ocurre durante cada uno de estos pasos. ¿Cuál es el objetivo principal de cada una de estas etapas?
8. Considere una red neuronal pequeña diseñada conceptualmente para entender su propagación, con 2 entradas (x_1, x_2), una capa oculta de 2 neuronas con activación ReLU, y una neurona de salida con activación Sigmoidea. Asigne valores numéricos arbitrarios sencillos (por ejemplo, 0.5, 1, -1) a los pesos (w) y sesgos (b) de esta red, y realice el cálculo manual paso a paso del *Forward Pass* para la entrada $x_1 = 1, x_2 = 0$. Detalle las combinaciones lineales (z) y el resultado de la activación (a) en cada capa.
9. Diseñe una red neuronal capaz de computar la función XOR de dos inputs binarios. Defina los pesos de cada conexión para que la red compute la función para todo posible combinación de datos de entrada.

10. Las librerías modernas de Deep Learning, como PyTorch, facilitan el cálculo complejo de los gradientes encadenados utilizando el módulo de Diferenciación Automática (*AutoGrad*). Explique conceptualmente qué es un Grafo Computacional (*Computational Graph*) y por qué es la pieza fundamental que permite a estas librerías entrenar cualquier arquitectura de red sin calcular las derivadas explícitas a mano.

Optimización, Hiperparámetros y Regularización

11. Contraste las ventajas y desventajas de las estrategias de optimización *Batch Gradient Descent* (usando todo el dataset) y *Stochastic Gradient Descent* puro (usando una sola muestra por iteración). Explique por qué en el entrenamiento práctico de redes neuronales se suele optar por un punto intermedio conocido como *Mini-Batch*.
12. Los algoritmos de optimización avanzados modernos, como ADAM, buscan superar limitaciones de SGD como el efecto de zig-zag (*ravine*) o el estancamiento en mesetas. Explique conceptualmente qué ventajas particulares aportan la inercia (*Momentum*) y el uso de un *Learning Rate Adaptativo* para cada parámetro.
13. Al configurar el entrenamiento de una red, adaptar la tasa de aprendizaje global (*Learning Rate*) en el tiempo suele ser beneficioso utilizando un *Scheduler*. Mencione al menos dos estrategias comunes y describa cómo operan a medida que transcurren las épocas.
14. La técnica de *Dropout* actúa como un regularizador durante el entrenamiento apagando al azar ciertas neuronas para evitar que aprendan patrones "de memoria". ¿Qué ocurre específicamente con la red al momento de realizar la inferencia y por qué es crucial utilizar la instrucción `modelo.eval()` en PyTorch en esta etapa?
15. A pesar de definir la arquitectura y el optimizador, las redes neuronales son muy sensibles a los rangos de los inputs. Justifique por qué es importante aplicar *Input Scaling* (estandarización) a las variables de entrada. Complementariamente, explique el rol que juega la capa de *Batch Normalization* para combatir el corrimiento de la distribución de los datos dentro de las capas ocultas de la red.